UNIT – II

Software Requirements :- Functional and non functional requirem, User requirements, System requirements, Interface specification, the Software requirements doumet.

Requirements Engineering Process :- feasibility study, Requirements elicitation and analysis, Requirements validation, Requirements management.

System models : context models, Behavioural models, Data models, Objet models, Structured models.

(I) Functional and Non functional requirements :

(i) Functional requirements :

* The functional requirements for a system describe what the system should do.
* hao system react to particular inputs, hao system should behave in particular situations. (Some cases it may also say what system should not do)

(ii) Non-functional requirements :

* Non-functional requirements are the constraints on services offered by the system (timing constraint, development procen, constraints, standard constraints).
* These requirements apply on the system as a whole rather than individual system.

1) Functional requirements :-

* Example of mental health care patiet management system (MHC-PMS) maintains information about patients receiving treatment for mental health problems

1. User shall able to search the appointments list for all clinics.
2. System shall generate each day, for each clinic, a list of Patients who are expeited to attend appointments that day.

3). Each staff using the system should uniquely identified by his/her eight digit employee number.

* Above all functional user requirements defines specific facilities
* system developer to interpret ambiguous requirement in a way that simplifies its implementation.

Eg: user shall search the appointment (1st requirement)

patients with mental health problem are sometimes confused, ie. they may have appointment in one clinic but actually go to a different clinic. (bcoz if appointment is there they are recorded as attended irrespective of clinic)

* so system developer implement in another way first choose a clinic and then carry out search.
* so requirements should be complete (all services required by user should be defined) and consistent. (requirements should not have contradictory definitions).

Problems :-

Practically for large complex systems it is impossible to achieve requirements consistency and completements.

reason :- 1) easy to make mistakes & omission when writing specification.

2) Many stakeholders in a large system, each of them have different inconsistent needs.

Stakeholder (a person or role that is affected by the system in some way)

Solution :- inconsistente requirements are included in the specification (problem only arise through deep analysis or after system is delivered to the customer)

2) Non-functional requirements

* Non-functional requirements, not directly concerned with the specific services delivered by the system.
* They may relate to emergent system properties such as reliability, response time, store occupancy, security, safety, interoperability etc.
* They define constraints on system implementation (I/O device capabilities, interface with other system)
* Non-functional requirements such as performance, security or availability usually specify constraint characteristics of a system as a whole.
* failing to meet non functional requirements means whole system is unusable.

Ex: 1) aircraft system if doesn't meet reliability requirements will not be certified as safe for operation.

2) Embedded control system fail to meet its ~~operation~~ performance requirements the control functions will not operate correctly.

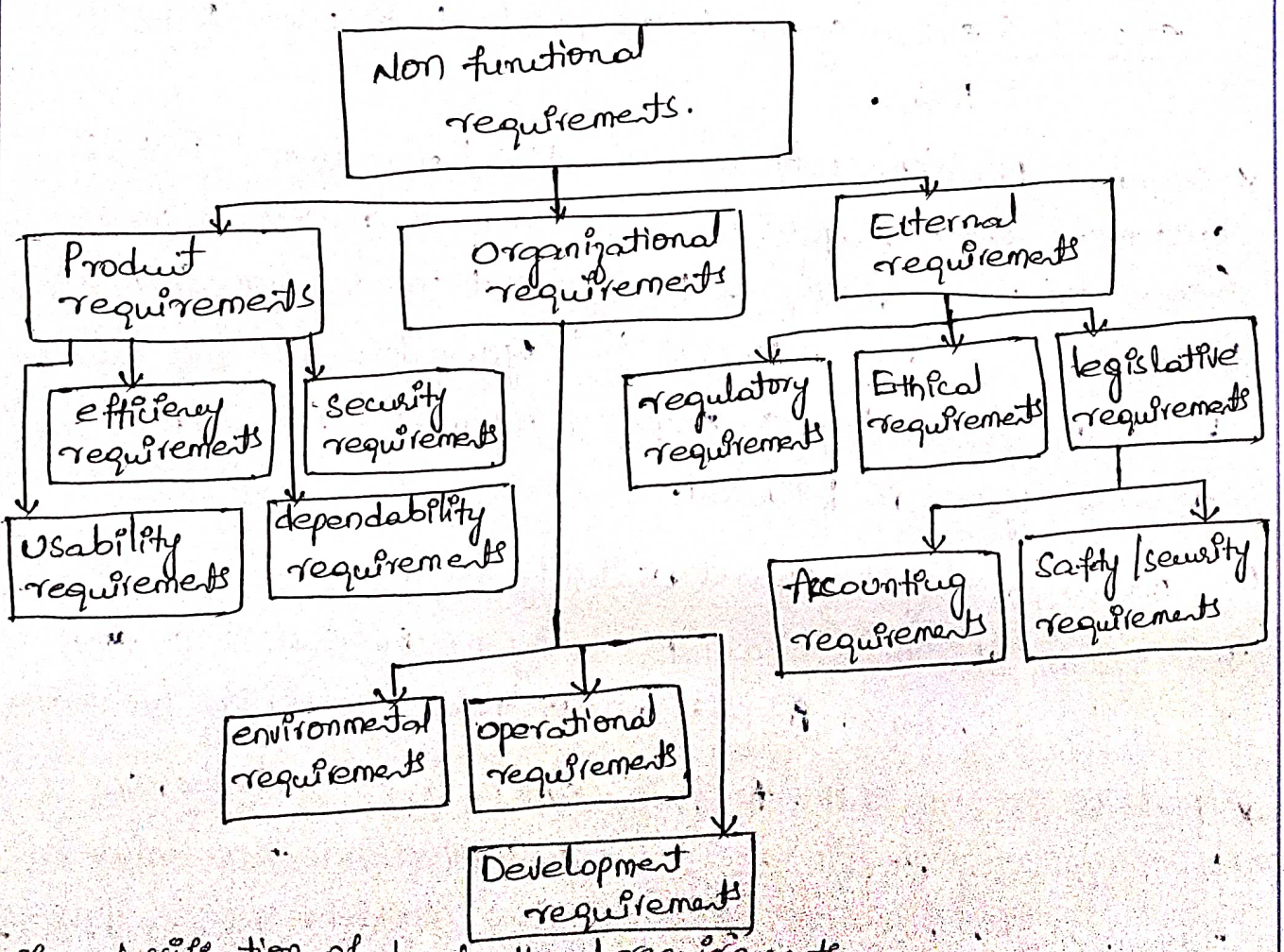


Fig: Classification of Non-functional requirements

Product requirements :- Specify the behaviour of the software
Ex: performance requirements (how fast system must execute, how much memory it requires), reliability requirements (acceptable failure rate), security requirements & usability req.

Organizational requirements : these requirements are system requ derived from policies and procedures in customer & developer organization.

Ex: operational : define how the system will be used
development : Process standard to be used
environmental : specify operating environment of the system.

External requirements : covers all requirements that are derived from factors external to the system and its development process.

Ex: regulatory : what must be done for the system to be approved for a use.
legislative : System operates within the law, it is give assurance
ethical : ensure system is acceptable to its users & general public.

* common problems with non-functional requirements is that customers often propose these requirements as general goals (ease of use, ability of the system to recover from failure)
* goals are of good intensions but system developer face problems bcoz. subsequent dispute, scope of interpretation is seen by him after the system is delivered.

Ex: manager might express usability requirements.
The system should be easy to use by the medical staff and should be organized in such a way that user errors are minimized
* but it is impossible to verify system goal objectively, so by converting usable requirements to testable non-functional requirement. we can atleast include software instrumentation.

to count errors made by users when testing the system. Medical staff shall be able to use all the system functions after proper training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use.

* So, if possible write non-functional requirements, it can be quantitatively measured and objectively tested.

* Metrics to specify non functional requirements.

Speed	-	Processed transactions event response time Screen refresh time
Size	-	1M bytes Number of Rom chips
Easeofuse	-	Training time Number of help frames.
Reliability	-	mean time to failure Probability of unavailability Availability.
Robustness	-	Time to restart after failure Percentage of events causing failure.
Portability	-	percentage of target dependent statements. No. of target systems

II) *** The Software Requirements document (SRS Document)

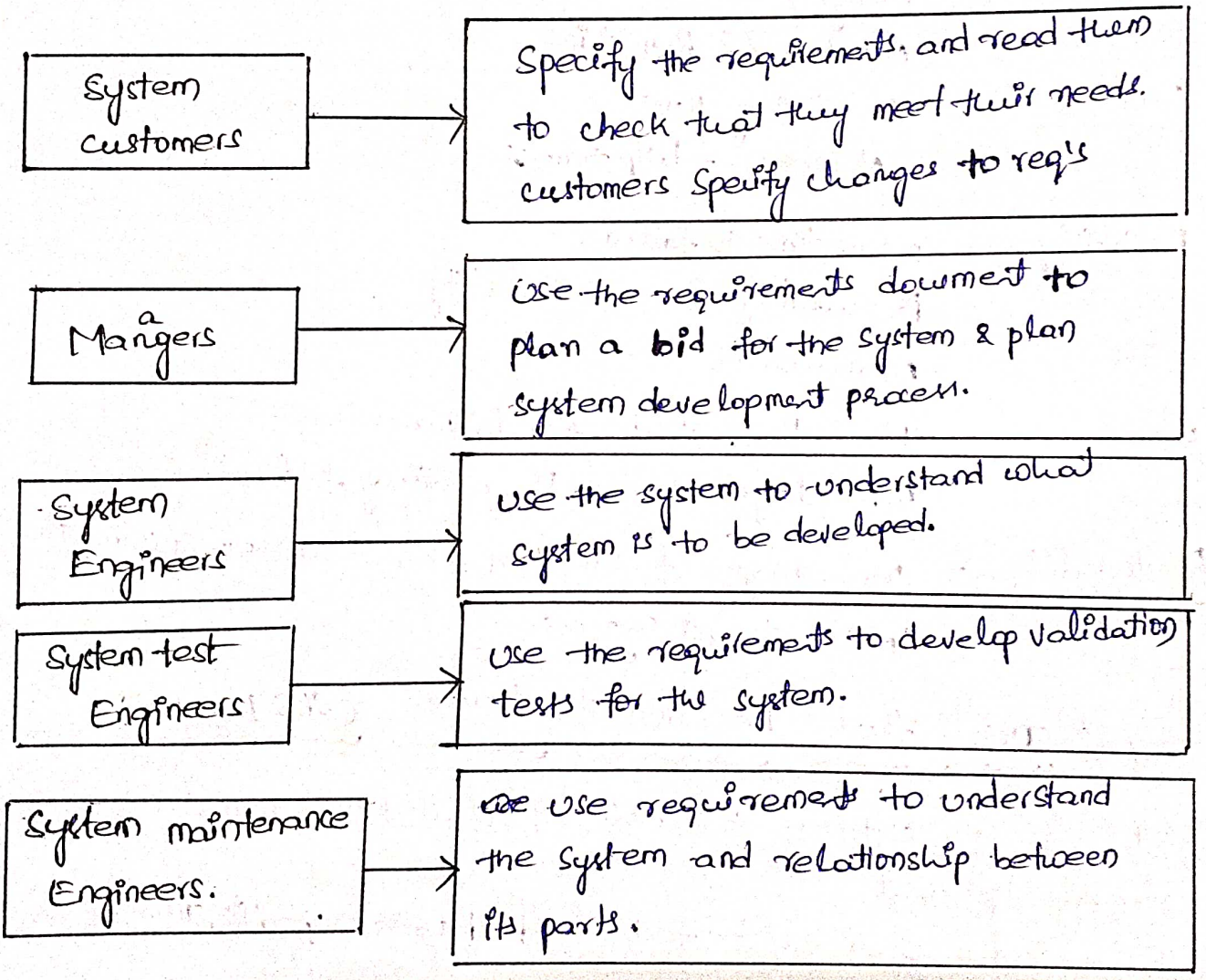
* SRS is an official statement of what the system developers should implement
* It includes user requirements and detailed specification of the system requirements
* If there are large number of requirements, the detailed system requirements may be presented in separate document.

* requirements doument is essetial when outside contractor is developing software system. But agile development says requirements change rapidly. So requirements doument become outdated as soon as it is written, causing waste of effort.

* Instead of that prepare doument using extreme programming collect requirements incrementally and write them as user stories, the user then prioritizes requirements for implementation.

Ex: Business systems.

* requirements doument has a diverse set of uses, ranging from seniors management to the engineers responsible for developing the software.



* Different types of users means the requirements doument has to be a compromise between communicating the requirements to customers, defining requirements in detail for developers & testers including possible system evolution.

* Level of detail in requirements daument depends on the type of system that is being developed and development process used.
* Critical systems needs detail daument becoz safety & security is analyzed.
* System developed by separate company than system specifications need to be detail.
* If Individual Iterative process is used then requirements daument can be less detail and any ambiguity can be resolved during development process.

(i) IEEE Standard for requirements daument.

Chapter	Description.
Preface	– Define expected readership of daument, describe history, include logical basis for creation of new version & summary of changes made in each version.
Introduction	– Describe need for the system, describe systems functionalities & explain how they work with other systems, how systems fits in overall business; objectives of the organization.
Glossary	– Define technical terms used, do not make any assumptions about the experience.
User requirements Definition	– * Describes services provided by users * Non functional req * Description use natural language, diagrams or other notations are used for representing non functional requirements. * Product and process standards must be specified.
System architecture	– * High level overview of system architecture * Distribution of functions across system modules * Architectural components that are reused are highlited

System requirements Specification – functional & non functional in detail further detail add to non functional interface to other system may be defined.

System models – set out one or more system models showing relationship b/w system components & system & its environment.

System Evolution – Anticipated changes due to H/w evolution, changing user needs.

Appendices – H/w & Database descriptions related to the application which is being developed.
Ex: optimal configuration of the system, Logical organization of data ~~based~~ used by the system & relationship b/w data.

Index – Normal alphabet index, index of diagrams index of functions etc.

Domain Requirements:-

* derived from the application domain of the system rather than from the specific needs of system users.
* They include specialized domain terminology or reference to domain concepts.
* Set out how the configuration is carried out becoz these requirements are specialized, so software engineers often find it difficult to understand how they relate to other system requirements.
* These requirements are important becoz they often reflect fundamentals of application domain, if these req's are not satisfied then it is impossible to make the system work satisfactorily.

E1: There shall be a standard user interface to all databases that shall be based on Z39.50 standard.

=

(III) User Requirements:-

* These requirements derive the functional and non functional req's so that they are understandable by system users without detailed technical knowledge.
* Should specify external behavior of the system and avoid system design characteristics.
* write the req's in simple language with simple tables & forms and intuitive diagrams.

Problems with natural language:-

1. Lack of clarity:- It is sometimes difficult to use language in a precise and unambiguous way without making the document difficult to read.

2. Requirements confusion:- functional & non functional req's goals and design information may not be clearly distinguished.

3. Requirements Amalgamation:- several req's may be expressed together as a single requirement.

E1: LIBsys shall provide financial accounting system that maintains records of all payments made by users of the system, system managers may configure the system so that regular users may receive discounted rates.

This example includes both conceptual & detailed information. i.e, it should be an accounting system as an inherent part of LIBsys also includes detail that accounting system should support documents of regular LIBsys users.

* To minimize misunderstandings when writing user req's following simple guidelines can be followed.

1. Invent a standard format and ensure that all req's definition use the same format like
   → Initial requirement in bold face
   → state of logic basis with each user requirements
   → reference to more detail system requirement specification

2. Use language consistently:- always distinguish between mandatory (written as shall; ie system must support) and desirable requirements. (written with should not essential)

3) Use text highlighting (Bold, italic) to pick out key parts of the requirements

4) Avoid use of computer jargon (non technical terms)

=

(IV) SYSTEM REQUIREMENTS.

* System requirements are used as starting point for the system design.
* They describe external behavior of the system and its operational constraint
* Not concerned with how the system should be designed or implemented
** If a complex software system is developed, it is impossible to Provide all design & information.

Reasons:-

1)* You design an initial architecture of the system to help structure the requirements specification
System req's are ~~designed~~ organized accordingly to different subsystems that make up the system.

2) * Most cases system must interoperate with other existing systems

3) Use of specific architecture to satisfy non functional req's may be necessary (Because external regulatory needs to certify that the system is safe).

(i) Natural language specification:—

* Natural language is used to write system & user reqts. for system requirements natural language specification is can be confusing and hard to understand.

① Natural language understanding relies on the specification readers & writers using same words for same concept. which leads to misunderstandings becoz of ambiguity of natural lang

② Language is over flexible (you can say anything in different ways it is upto reader to findout whether requirements are same or different)

③ No easy way to modularize natural language requirements, it may be difficult to find all related requirements.

— Became of these problems natural language specification is prone to misunderstandings, and if any requirements discovered in the later stages leads to expensive to resolve.

=

Notations for requirements specifications:—

*Alternatives to natural language specification for specifying reqts are

1) Structured Natural language: This approach depends on designing standard form or templates to express the reqts specification.

2) Design description languages:— This approach uses a language like a programming lang but with more abstrait features to specify reqts by defining an operational model of the system.

3) Graphical Notations:— A graphical language, telt annotations is used to define the functional reqts. Usecase & sequence diagrams are commonly used

(4) Mathematical specifications:— Notations based on mathematical concepts such as finite state machine or sets.

(1) Structured Language Specifications

* Is a way of writing system req's where there is less freedom of all requirements written in a standard way.

Advantages:-

1) Expressiveness & understandability of natural language but ensure degree of uniformity.
2) limit the terminology and use templates to specify system req's
3) Incorporate control constructs derived from programming languages and graphical highlighting to partition the specification.

Ex:- Insulin pump system

Function : Insulin dose

Description : Dose of insulin if it is in safe zone 3 and ≠ units

Inputs:- current reading (r_2), previous two reading (r_0, r_1)

Source : current reading from sensor

Outputs:- Dose of insulin delivered.

Destination:- Maintain control loop

Action:- Dose is zero if sugar level is stable or falling or level is increasing but rate of increase is decreasing level is increasing but rate of increase is increasing then dose compute by dividing diff blw current sugar level and previous level by 4 and rounding the result, if result : zero the dose is set to minimum dose to deliver.

Requires: Two previous readings to compute sugar level rate of change

Precondition:- insulin reservior contains at least min allowed single dose.

Post condition:- r_0 is replaced r_1 then r_1 is replaced by r_2.

Side effects : None.

When standard form is used for specifying functional req's the following information should be included.

1. Description of the function or entity being specified
2. Description of i/p's and where these come from
3. " " o/p's and where they go to
4. Indication of what other entities are used
5. Description of action to be taken
6. If function is used pre-condition setting out what must be true before function called and post-condition specifying what is true after the function is called.
7. Description of side effects if any.

(V) Interface Specification:—

* All software systems must operate with existing systems that have already implemented & installed in an environment.
* If new & existing system should work together. Interface of existing system must be specified.

Three types of interfaces:—

1. Procedural interface:— where existing programs or such systems offer a range of services that are accessed by calling interface procedures (API's)
2. Data Structures:— that are passed from one sub system to another graphicals, data models are best notation.
3. Representation of data:— that have been established for an existing sub system (common in embedded, real time system) best way to describe these is probably to use a diagram of the structure with annotations explaining the functions of each group.

Ex: procedural interface definition defined in java

```
Interface PrintServer {
    // defines an abstract printerServer
    // requires interface printer, interface PrintDoc
    // Provides initialize, print, displayPrintQueue, cancelPrintJob,
                                                    switchPrinter.
```

```
Void initialize(printer P)
Void print (Printer P1, PrintDoc d);
Void displayPrintQueue (Printer P);
Void cancelPrintJob (Printer P1, PrintDoc d);
Void switchPrinter (Printer P1, printer P2, PrintDoc d);
} // printServer.
```

=.

(VI) ## Requirements Engineering Process.

Requirements Engineering Process includes 4 high-level activities.

1) Focus on assessing if the system is useful to the business (feasibility study)
2) Discovering requirements (elicitation and analysis)
3) Converting these requirements into some standard form (specification) and
4) checking that the requirements actually define the system that the customer wants (Validation)

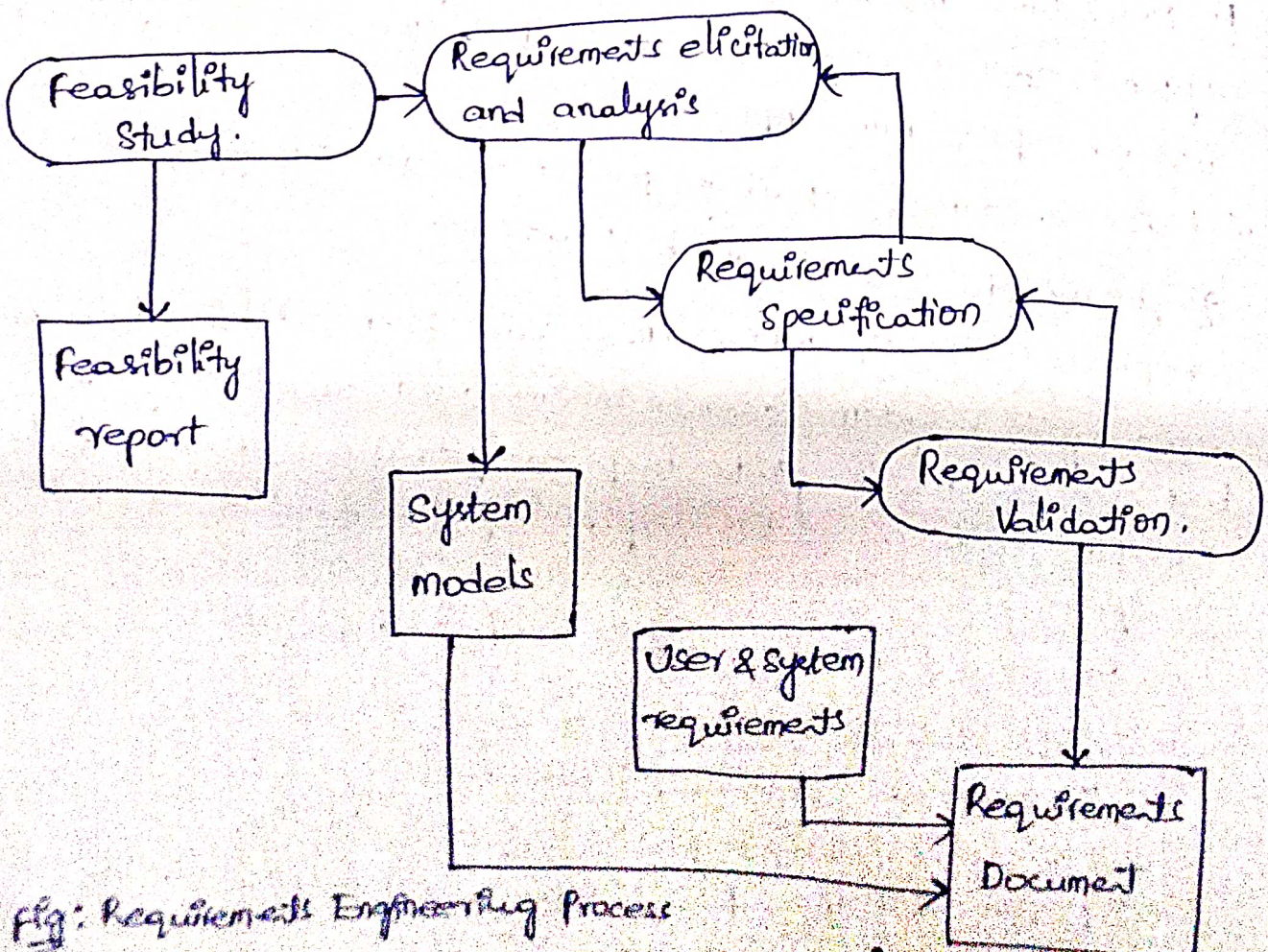
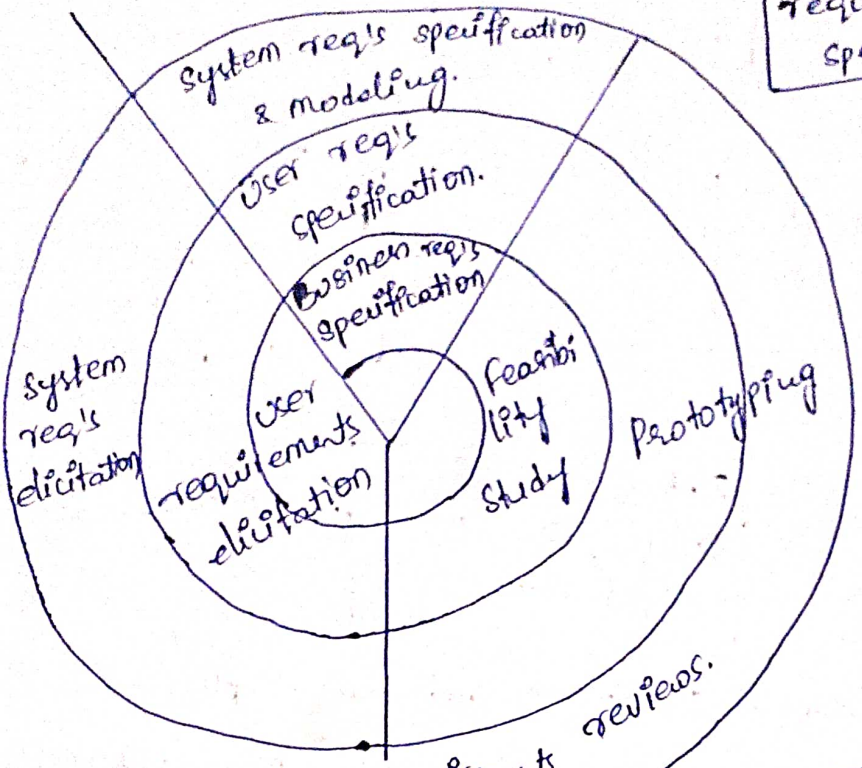


Fig: Requirements Engineering Process

Spiral model of requirements engineering process.



* people consider that, requirements engineering process is a way of specifying requirements using structured analysis method such as analyzing the system, and developing a set of graphical system models, such as use case model, structured method specificati of requirements belongs to only one part in requirements engineering. Still there is much more to be added for requirements engineering.

One of such activity is Requirement elicitation.

* It is a human centered activity.

(VII) *** Imp. **Requirements Elicitation and Analysis**

* After initial feasibility study, the next stage of requirements engineering process is requirements elicitation & analysis
* In this activity, SW engineers work with customers & system end-users to find out about the application domain,
* what services the system should provide, the required performance of the system, HW constraints and so on.

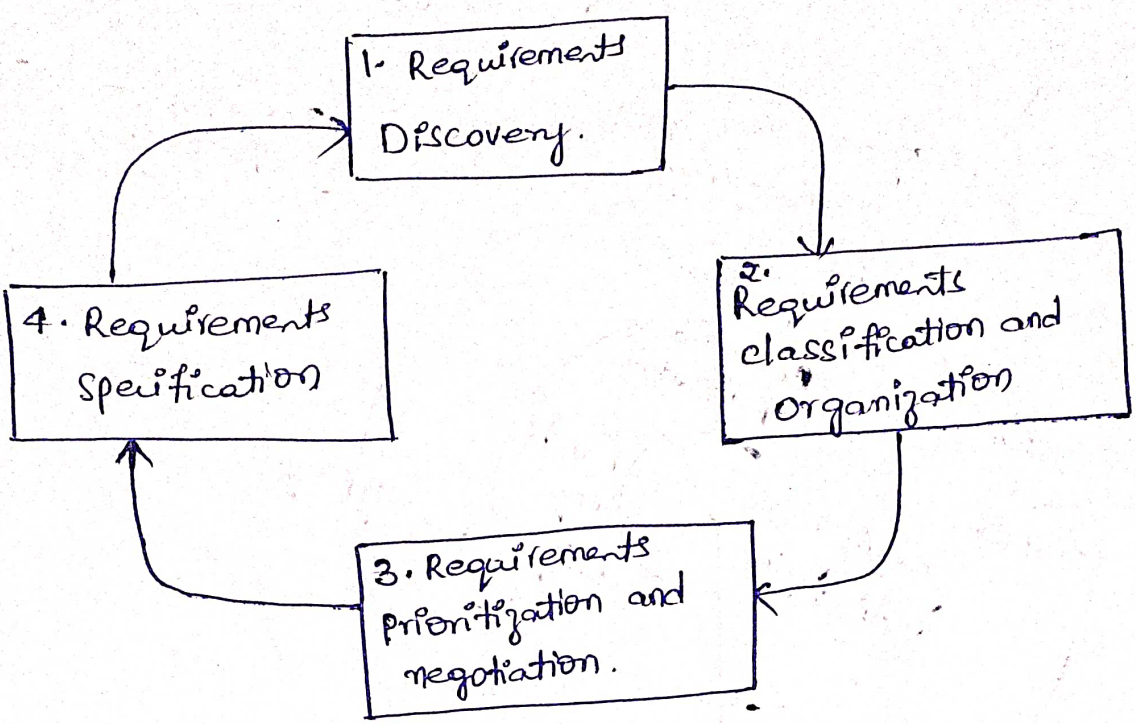


Fig: Requirements Elicitation and analysis process.

* Requirements elicitation and analysis may involve a variety of different kinds of people in an organization.
* A system stakeholder is anyone who should have some direct or indirect influence on the system requirements.
* An end user - who will interact with the system, and anyone else in the organization who will be affected by it.
* Other system stakeholders might be engineers who are developing or maintaining other related system, business managers, domain experts and trade union representatives.

The process activities are :-

1. Requirements Discovery :-

* This is the process of interacting with stakeholders of the system to discover their requirements.
* Domain req from stakeholders and documentation are also discovered during this activity.
* There are several techniques that can be used for requirements discovery.

2) Requirements classification and organization:—

* This activity takes the unstructured collection of requirements, groups related requirements, and organizes them into coherent clusters.

③ Requirements prioritization and negotiation:—

* When multiple stake holders are involved, requirements will conflict
* This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation.
* Usually stakeholders have to meet to resolve differences and agree on compromise requirements.

④ Requirements specification:—

* In this activity requirements are doumented and are provided as input to the next spiral.
* Formal or informal requirements doument may be produced.

→ Requirements elicitation and analysis is an iterative process with continual feedback from each activity to other activities.

* Process cycle starts with requirements discovery and ends with the requirements documentation. In each iteration some kind of improvement will take place by the analyst.

Understanding stakeholders requirements is difficult for several reasons:—

1. Stakeholders often they don't know what they want from a computer system, they ~~dont~~ may find difficult to articulate what they want the system to do, They may make unrealistic demands, because they don't know what is feasible what is not feasible.
2. Stakeholders in a system naturally express requirements in their own terms and with implicit knowledge of their own work. Requirements engineers without experience in the customer's domain, may not understand the requirements.

* Stakeholders are the people related to the system. They may be end-users, managers, external stakeholders, They all should certify the acceptability of the system.

Example: Stakeholders for a Bank ATM

1. current bank customers who receives services from the system
2. Representatives from other Banks
3. Managers of the bank branches (obtain management infor from system)
4. Counter staff (who involved in the day-to-day running of the system)
5. Database administration (integrate system with bank customer db)
6. Bank security managers (ensure system will not pose a security hazard)
7. Bank's marketing department (intrested to use the system for marketing)
8. H/w & S/w maintenance engineers (upgrading of h/w & s/w)
9. National banking regulators (system confirms banking regulations)

In addition to stakeholders, requirements may also come from the application domain and from other systems that interact with the system being developing.

* Requirement sources (stakeholders, domain, systems) can all be represented as system Viewpoints

(1) Viewpoints:- Viewpoints are a way of structuring requirements to represent the perspectives of different stakeholders. Stakehold may be classified under different viewpoints.

Types of Viewpoints:-

1) Interactor viewpoints
2) Indirect viewpoints
3) Domain viewpoints.

1. Interactor Viewpoints :— people or other systems that interact directly with the system.

* These viewpoints provide detailed system requirements covering the system features and interfaces.
* In an ATM, the customer's and the account database are interactor viewpoints.

2. Indirect Viewpoints :— stakeholders who do not use the system themselves but who influence the requirements.

* These viewpoints are more likely to provide high-level organizational requirements and constraints.
* In an ATM, management and security staff are indirect viewpoints

3. Domain Viewpoints :—

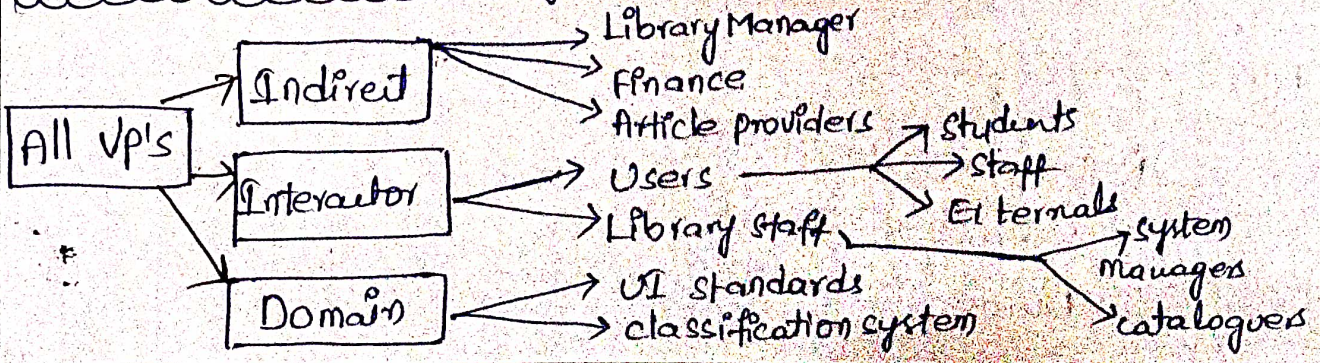
* Domain characteristics and constraints that ~~left~~ influence the requirements.
* These normally provide domain constraints that apply to the system.
* In an ATM, an example would be standards for inter-bank communications.

Viewpoints identification :—

Identify viewpoints using:

→ providers and receivers of the system services;
→ Systems that interact directly with the system being specified;
→ Regulations and standards.
→ Sources of business and non-functional requirements.
→ Engineers who have to develop and maintain the system.
→ Marketing and other business viewpoints.

LIBSYS viewpoint hierarchy :



(2) Interviewing :- Formal or informal interviews with system stakeholders are part of most req's engineering processes.

* Req's engineering team puts questions to the stakeholders about the system, that they currently use and the system to be developed.
* Requirements are derived from the answers to these questions.

Interviews may be of two types :-

1. closed interviews, where the stakeholders answers a predefined set of questions.
2. Open interviews, where there is no predefined agenda. The req's engineering team explore a range of issues with system stakeholders and hence develop a better understanding of the system

Interviews in practice will be like

1) Mix of closed and open-ended interviewing.
2) Interviews are good to conduct for better understanding of stakeholders views about the system. working with current system, issues with the current system.
3) Interviews are not good for ~~what they need from~~ understanding system domain requirements.

* Domain requirements completely related to technical information about the system. so, if interviews happened with stakeholders regarding domain req's, req's engineers cannot understand specific domain terminology.

Effective Interviewers must have two characteristics

1. Interviewers will not be open-minded, willing to listen to stakeholders and should not have pre-conceived ideas about req's
2. They should prompt the interviewee with a question or a proposal and should not simply expect themselves to respond to a

question such as 'what do you want"

(3) Scenarios :-

* Scenarios are real life examples real time examples than to abstract description. They can understand and analyze a scenario of how they might interact with a s/w system.
* requirements engineer gather information for this discussion to formulate the actual system requirements.
* Each scenario covers one or more possible interactions. Several scenarios have developed each of which provides different types of information at different levels of detail about the system.

Scenarios include :-

→ what the system and users except when the scenarios starts
* Description of normal flow of events
* Description of the starting situation
* Description of what can go wrong.
* Information about other concurrent activities.
* Scenarios may be written as text, implemented by diagrams, screen shots and so on.

(IX) Ethnography :-

* Software systems do not exist in isolation (separate) they are used in a social and organizational context and software system requirements may be derived or constrained by the context
* Satisfying these social and organizational requirements is often critical for the success of the system.

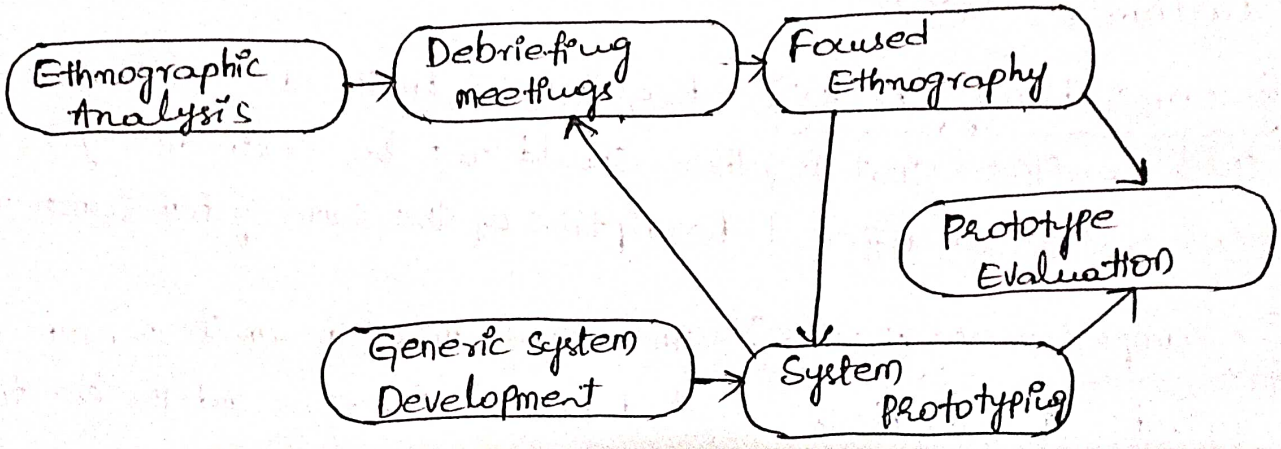
reason: Many s/w systems are delivered but never used becoz they do not take proper account of the importance of these requirements.

* Ethnography is an observational technique that can be used to understand social and organizational requirements.
* Value of ethnography is that it helps analysts discover implicit (internal) system requirements that reflect actual rather than formal processes in which people are involved.

Ethnography is effective at discovering two types of requirements.

* Requirements that are derived from the way in which people actually work rather than the way in process definitions say they ought to work.
* Requirements that are derived from cooperation & awareness of other people's activities. (Ex: air traffic controllers may use awareness of other controllers)

Ethnography and prototyping for requirement analysis.



* Ethnography can be combined with prototyping. The Ethnography informs the development of the prototype so that fewer prototype refinement cycles are required. Furthermore, the prototyping focuses the ethnography by identifying problems and questions that can then be discussed with the ethnographer.

=

(8X) Requirements Validation :- Requirements validation is the process of checking that requirements actually define the system that the customer really wants

* Requirements validation is important because errors in a requirements document can lead to extensive rework costs when these problems are discovered during development or after the system is in service.
* During the requirements validation process, different type of checks should be carried out on the requirements in the requirements document.
* These checks include :

1. Validity checks :- A user may think that a system is needed to perform certain functions. It checks "Does the system provide the services or functions which really best support the customer's need?

2. Consistency checks :- Requirements in the document should not conflict. That is, there should not be contradictory constraints or different descriptions of the same system function.

3. Completeness checks :- The requirements document should include requirements that define all functions and the constraints intended by the system user.

4. Realism checks :- Using knowledge of existing technology, the requirements should be checked to ensure that they can actually be implemented.

5. Verifiability :- To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable.

There are a number of requirements validation techniques available.

1. Requirements reviews :- The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.

2. Prototyping :- In this approach, an executable model of the system in question is demonstrated to end-users and customers.

3. Test-case generation :- Requirements should be testable. If the tests for the requirements are devised as part of the validation process, then often reveals requirements problems.

=

(XI) Requirements Management

The requirements for large software systems are always changing. The reason for this is, during the software process, the stakeholder's understanding of the problem is constantly changing. The system requirements must

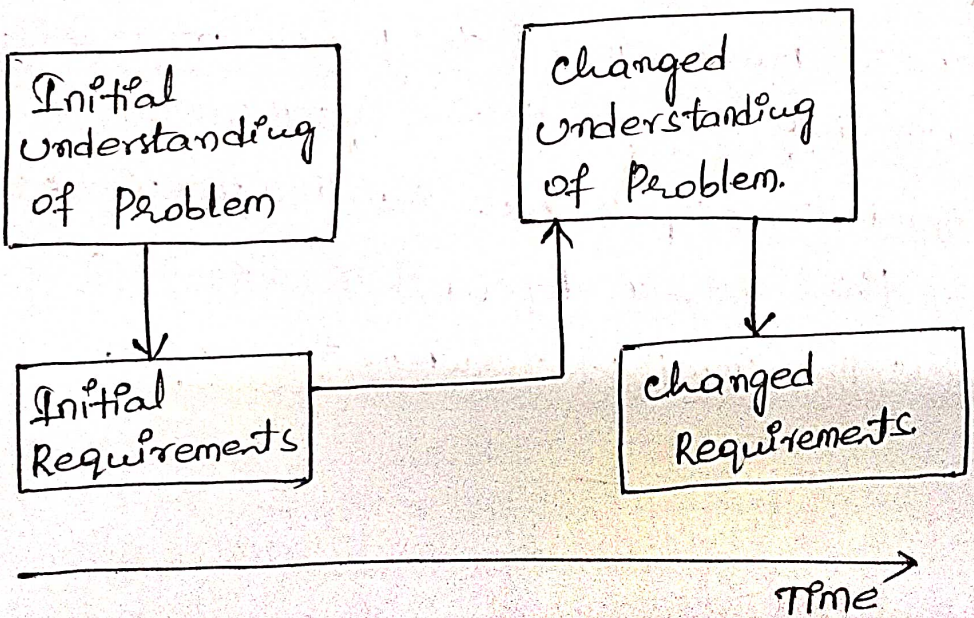


Fig: Requirements evolution.

=

then also evolve to reflect this changed problem view.

There are several reasons why change is inevitable:

1. The business and technical environment of the system always changes after installation. New hlw may be introduced, it may be necessary to interface the system with other systems, business priorities may change, and new legislation and regulations may be introduced that the system must include.

2. The people who pay for a system and the users of that system are rarely the same people. System customers impose requirements because of organizational & budget constraints. These may conflict with end-user requirements after delivery, so new features may have to be added for user support.

3. Large systems usually have a diverse user community. Many users having different requirements and priorities that may be conflicting. The final system requirements discovered would be that the balance of support given to different users has to be changed.

→ Requirements management is the process of understanding and controlling changes to the system requirements. There is a need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirement changes.

(i) Requirements change Management :-

Requirements change management should be applied to all proposed changes to a system's requirements after the requirements doument has been approved.

* change management is essential because you need to decide if the benefits of implementing new requirements are justified by the costs of implementation.

* There are three principal stages to a change management Procen.

1) problem analysis and change specification : The process starts with an identified requirements problem or, Sometimes, with a specific change proposal. During this stage, the problem or the change proposal is analyzed to check that it is valid.

2. change analysis and costing :- The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost of making the change is estimated both in terms of modifications to the requirements doument and if appropriate, to the system design and implementation. once the analysis is completed, a decision is made whether or not to proceed with the requirements change.

3. change implementation : Requirements document should be organized in such a way that it allows to make chages to it without extensive rewriting and reorganization.

* As with programs changeability in douments is achieved by minimizing external references and making the doument section as modular as possible.

=

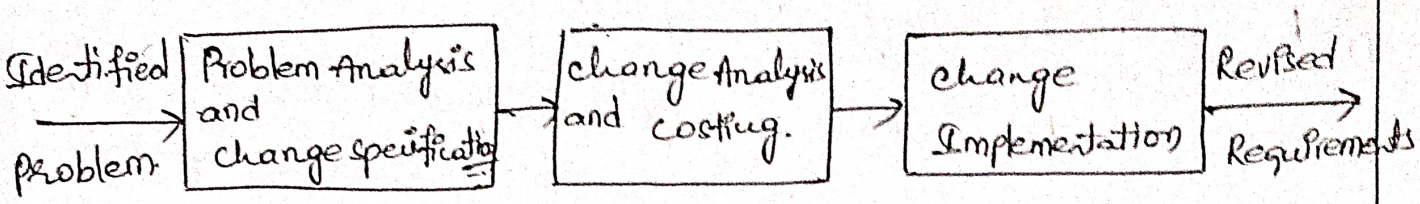


Fig: Req's change Management

(ii) Requirements management planning :-

planning is an essential first stage in requirements management process. The planning stage establishes the level of requirements management detail that is required.

* During the requirements management stage, you have to decide on:

1. Requirements identification :- Each requirement must be uniquely identified so that it can be cross-referenced with other requirements and used in traceability assessments.

2. A change management process :- This is the set of activities that assess the impact and cost changes.

3. Traceability policies :- These policies define the relationships between each requirement and between the requirements and the system design that should be recorded.

4. Tool support Requirements Management involves the Processing of large amounts of information about the requirements. Tools that may be used range from specialist requirements management systems to spreadsheets and simple database system